Compiladores e Intérpretes
Pautas Etapa 4
Analizador Semántico Parte 2

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

Introducción - Aclaraciones

- El desarrollo de esta **etapa** del proyecto se **corresponde** con los **de Chequeo de Sentencias** de la materia
- Te recomendamos que **hayas mirado y tengas frescos** todos los temas allí tratados **antes de arrancar** el diseño/desarrollo de esta etapa
- También te **esencial** mirar el documento de **MiniJava-Semantica Completo**
- En los ejemplos de los diseños de esta presentación vas a ver que hay “...”, eso quiere decir que es muy posible que a el diseño tenga algunas/varias cosas mas

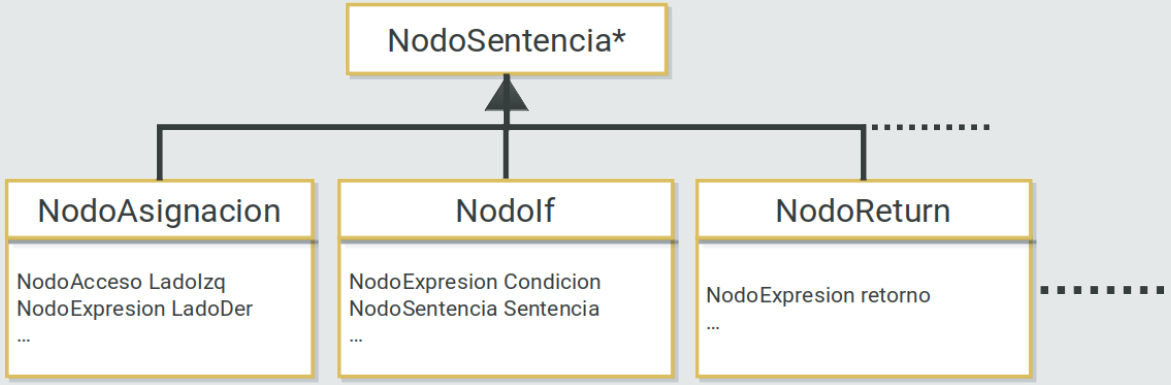
Introducción

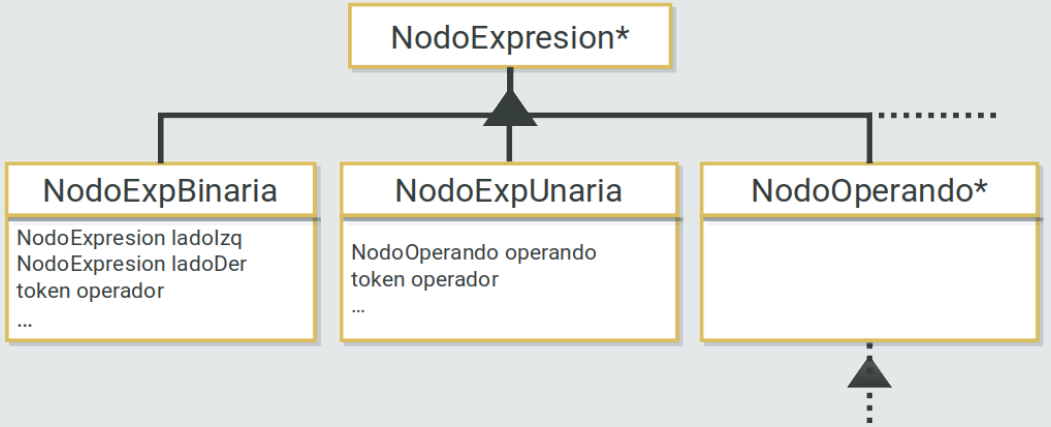
- En esta **etapa** hay que **diseñar e implementar** un **el Chequeo de Sentencias del Analizador Semántico** para **MiniJava**
- Para esto abrá que extender al **Analizador Sintáctico** de la etapa anterior para que cree que los **ASTs** de los métodos/constructores y los asocie a sus entradas en la Tabla de Símbolos (**TS**)
- El **Chequeo de Sentencias** se realizará directamente sobre esos **ASTs** en la **TS**
- Ademas, de manera similar a la etapa anterior va ser necesario un **Modulo Principal** que:
  - Manejará la **interfaz** con el **usuario** y
  - **Reportará los errores** detectados por el Semántico, el Sintáctico y el Léxico
  - En caso de no haber errores indicará que el análisis fue exitoso

Diseño de los ASTs y el Chequeo de Sentencias

Diseño de la Estructura de los ASTs

- Para diseñar la ASTs te recomendamos mirar 4.5.1 y 4.5.2
- Primero hay que **identificar y clasificar** los **tipos de nodos** del AST
  - La idea general es que **cada tipo de sentencia y expresión** se corresponderá con un **tipo de nodo** (el cual a su vez será una **clase**)
  - En las **clases** los **atributos** se usan para representar la **información** y los **hijos** que el tipo de nodo puede tener
  - Es importante también identificar la relación "**es-un**" (modelado con herencia) entre los tipos de nodos (e.g. una Expresion Binaria es una Expresión, o un Literal es un Operando)





Diseño de la Construcción de los ASTs

- Los **ASTs** se **construyen** a medida que se realiza el **análisis sintáctico**
  - Para **diseñar** que hacer usamos una **EDT**
    - Esta EDT debe realizarse sobre la **versión de la gramática** de MiniJava que usaron para **implementar** el **analizador sintáctico**
    - La idea es que las **acciones** de la EDT vayan **construyendo los ASTs** a medida que vamos **reconociendo sintacticamente** cada sentencia
    - Una vez **armado el AST del bloque** de un **método/constructor** se asocia a la entrada del subprograma en la **TS**

Diseño del Chequeo de Sentencias

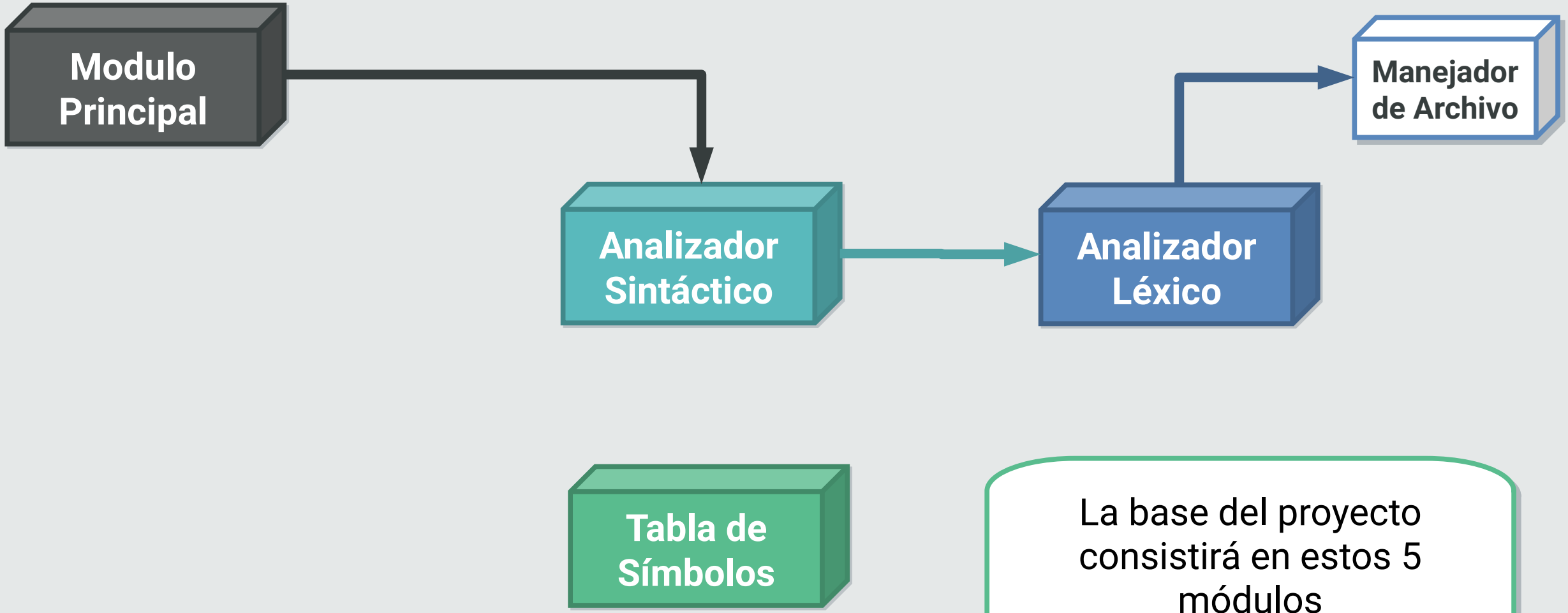
- El Chequeo de Sentencias consiste de **dos tareas** principales que se realizan en paralelo
  - **Resolución de Nombres**
  - Control de la **correctitud** (incluyendo el **chequeo de tipos**) de todos los nodos de cada AST
- Estas tareas se pueden **diseñar** como funciones/**métodos** directamente sobre la **TS** y las clases de las **Nodos**

Diseño del Chequeo de Sentencias

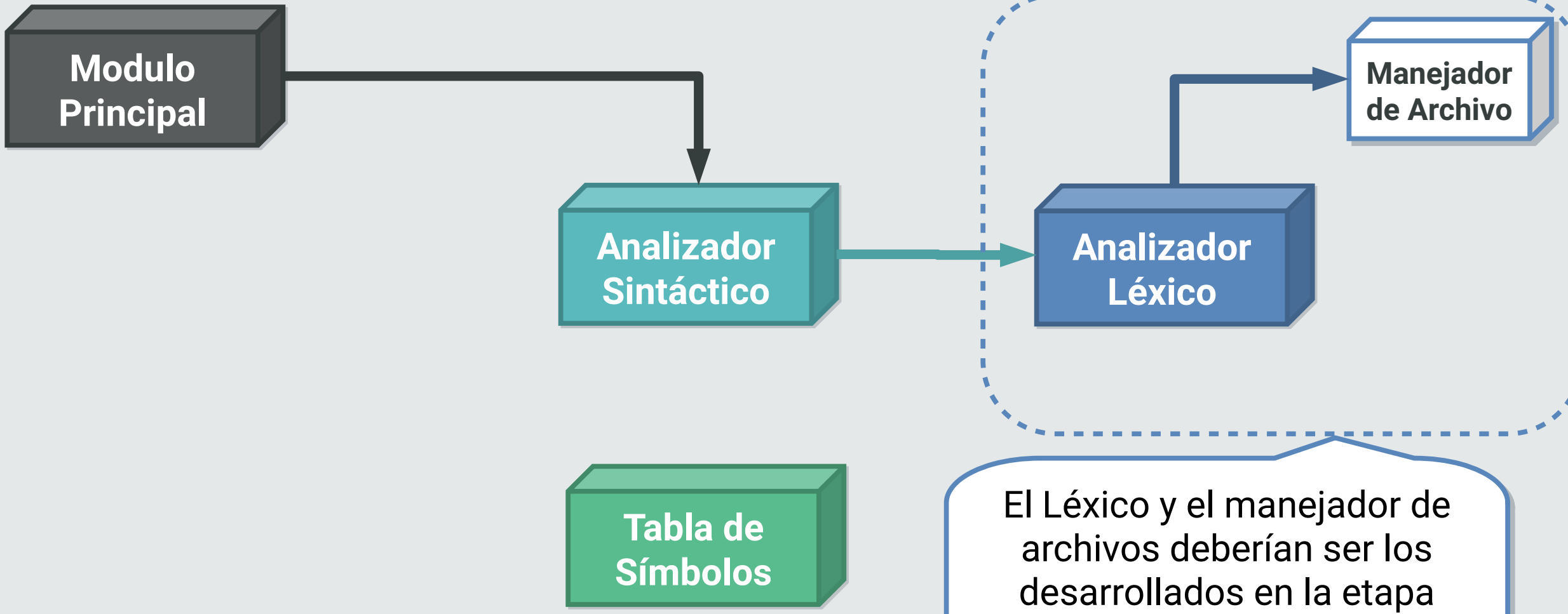
- Para diseñar esta tarea la clave es valernos de la **Delegación** y las **Responsabilidades** de cada entidad
  - La idea es que cada nodo **controla** lo que le **corresponde** (su **responsabilidad**) y **solicita** a cada hijo que se controle (**delega** el control)

Estructura y Funcionalidad del Software a Desarrollar

Esquema General

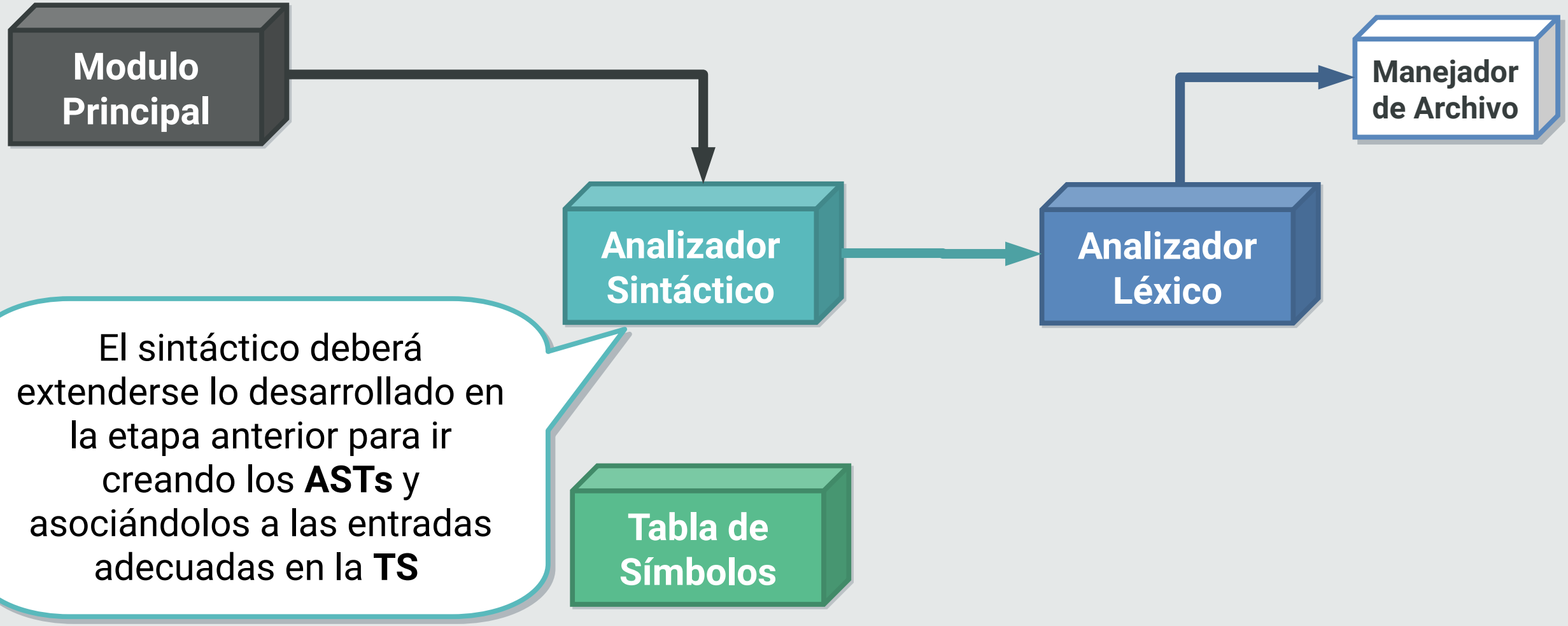


Esquema General

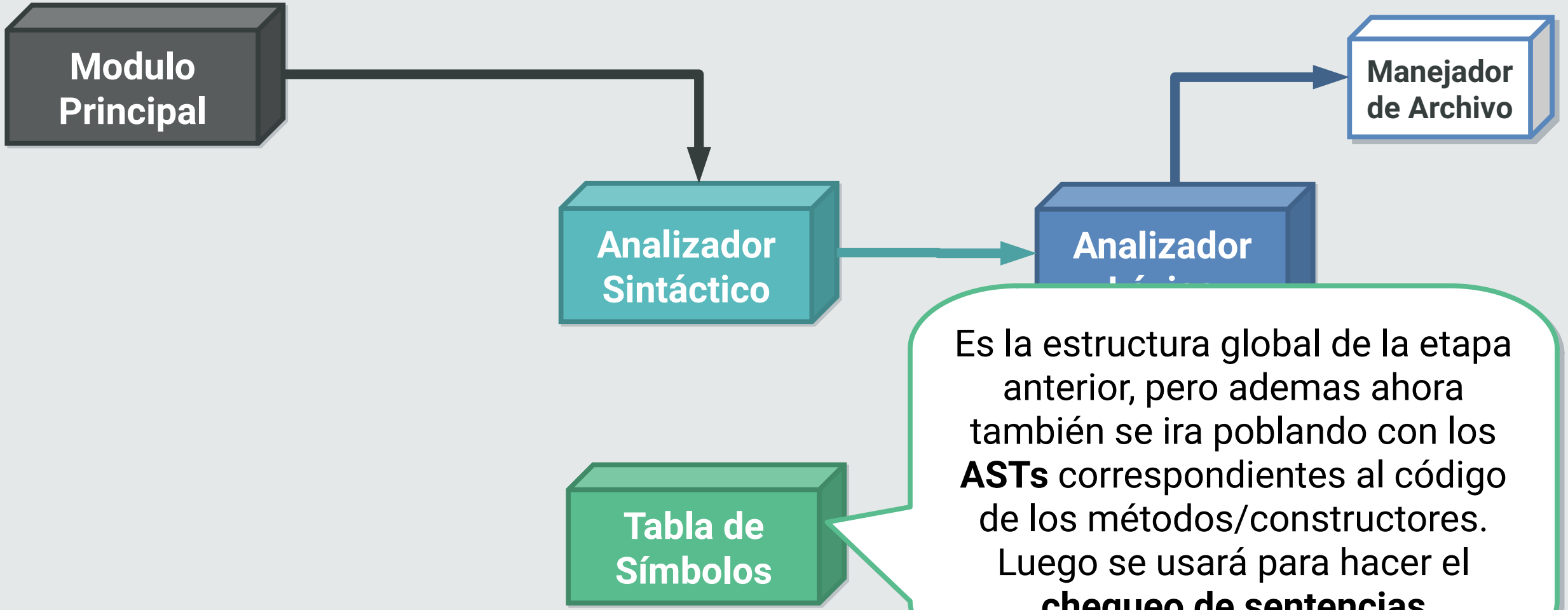


El Léxico y el manejador de archivos deberían ser los desarrollados en la etapa anterior (a lo sumo con sus bugs corregidos)

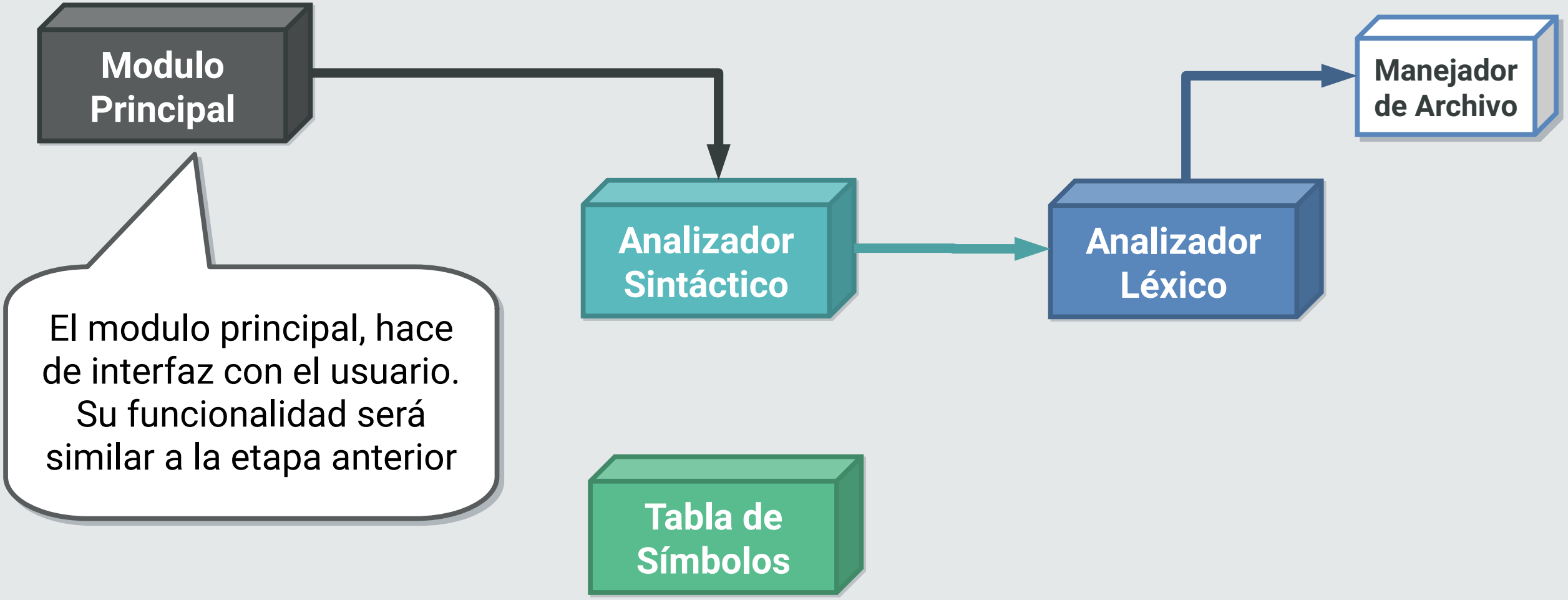
Esquema General



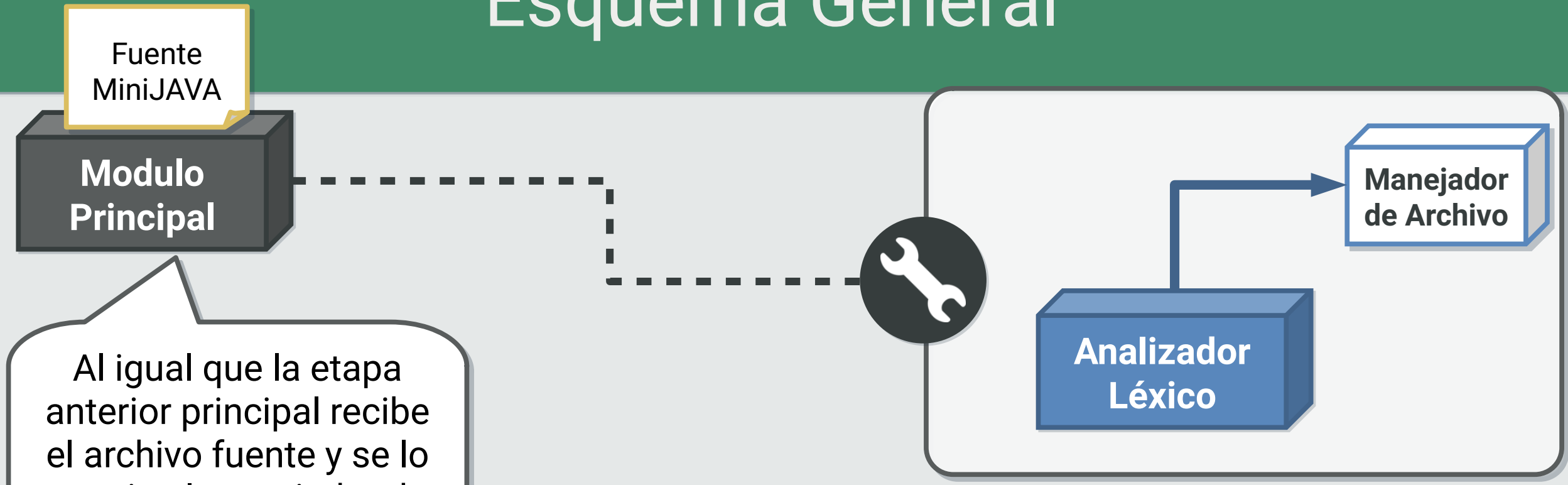
Esquema General



Esquema General



Esquema General

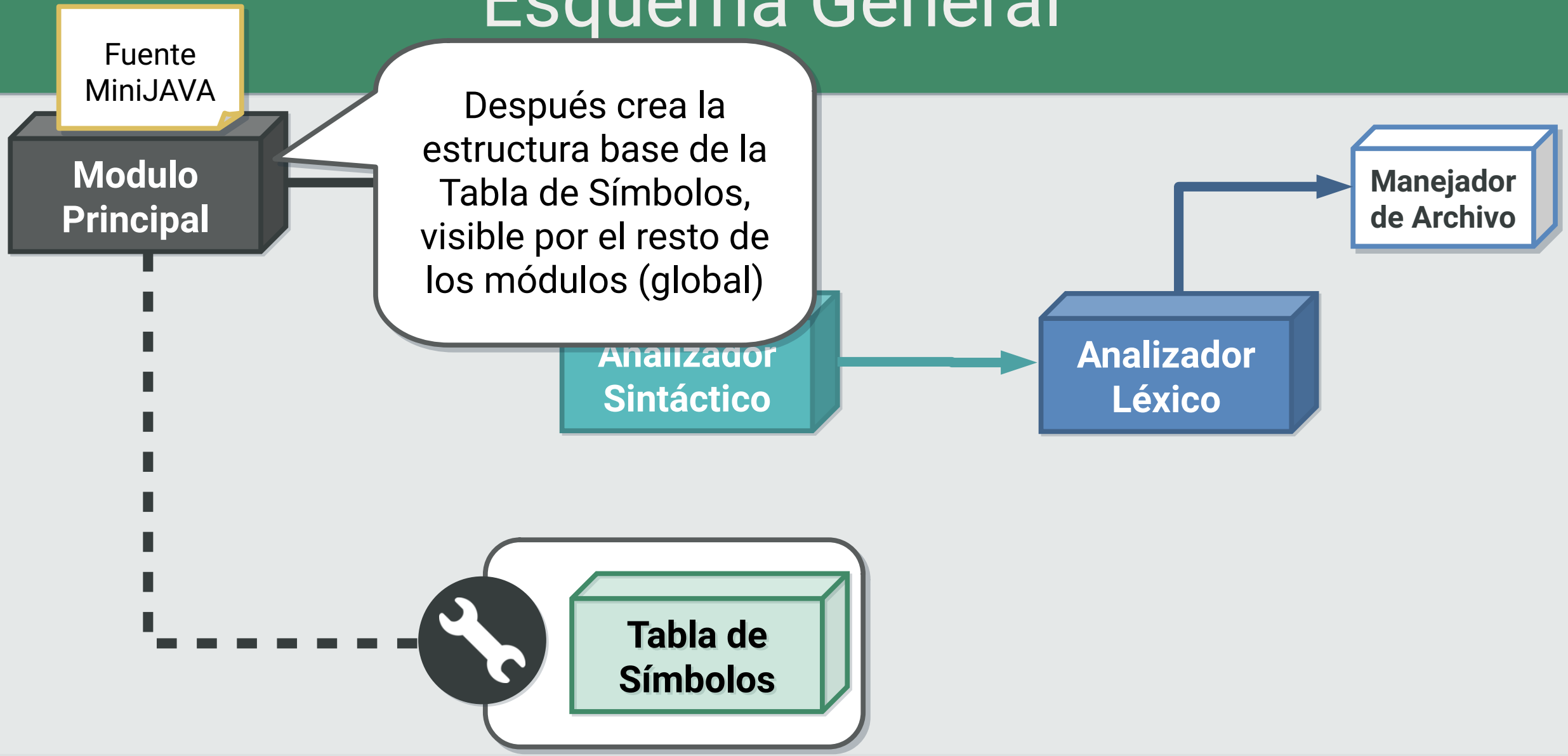


Al igual que la etapa anterior principal recibe el archivo fuente y se lo asocia al manejador de archivos y este al léxico

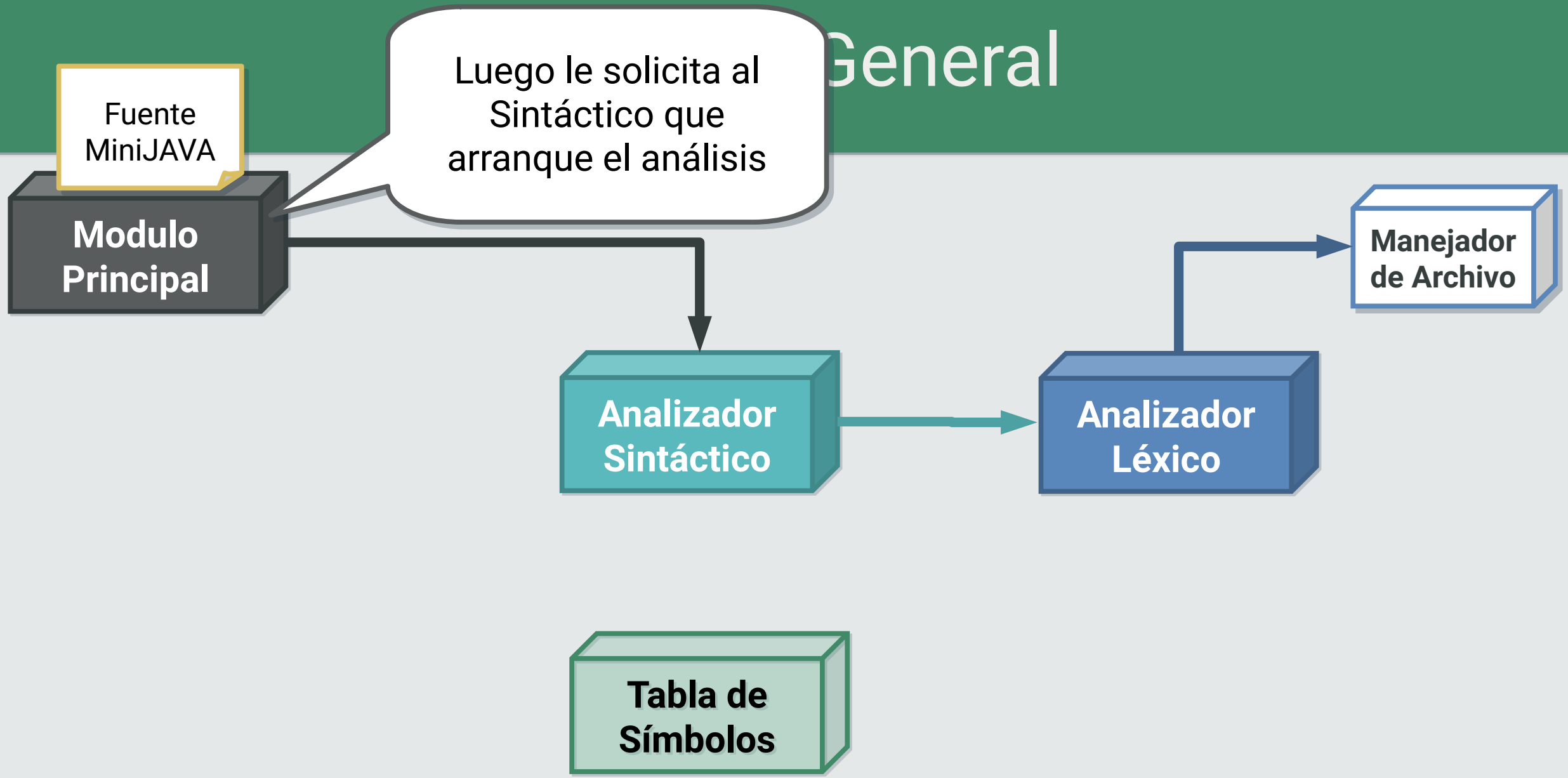
Esquema General



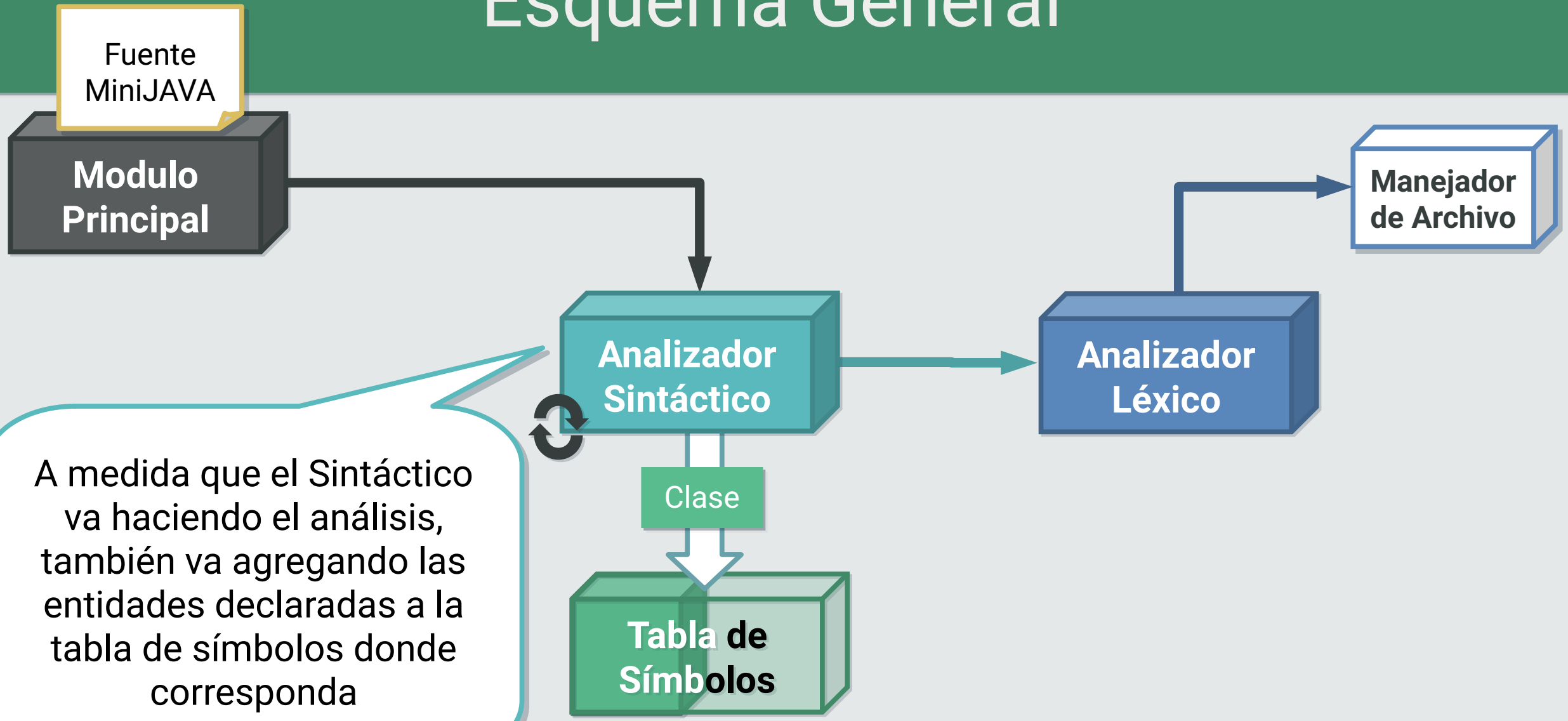
Esquema General



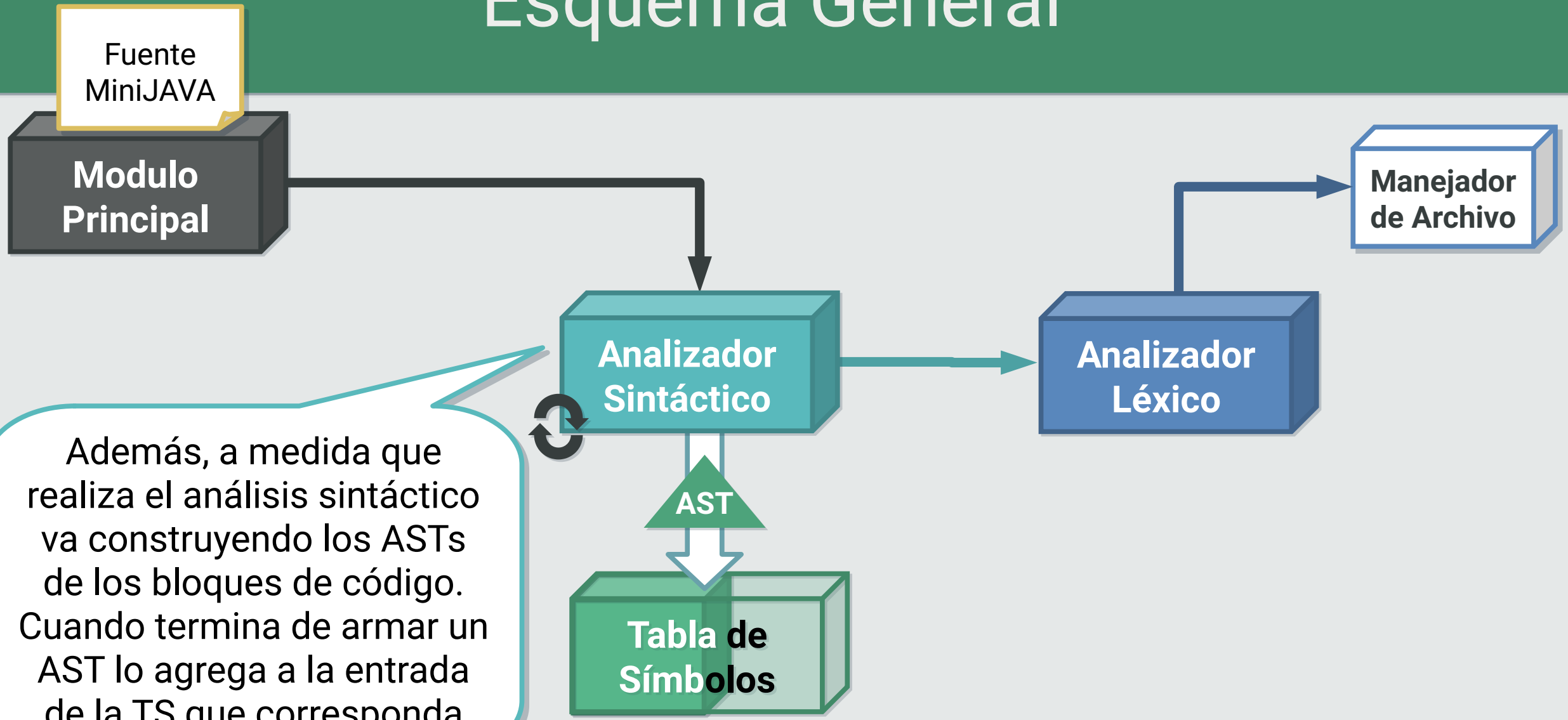
General


Fuente MiniJAVA
Luego le solicita al Sintáctico que arranque el análisis
Modulo Principal
Analizador Sintáctico
Analizador Léxico
Manejador de Archivo
Tabla de Símbolos

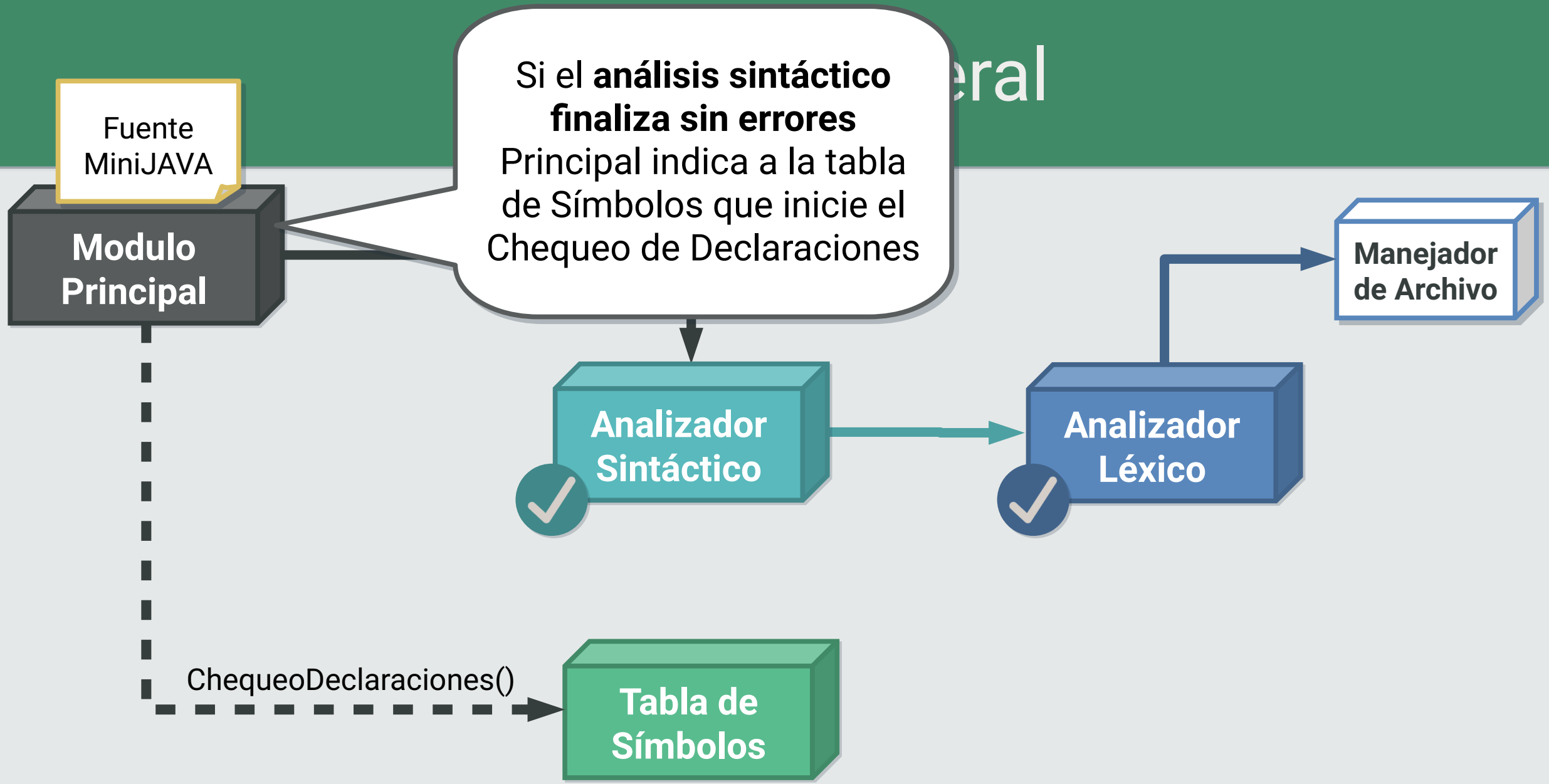
Esquema General



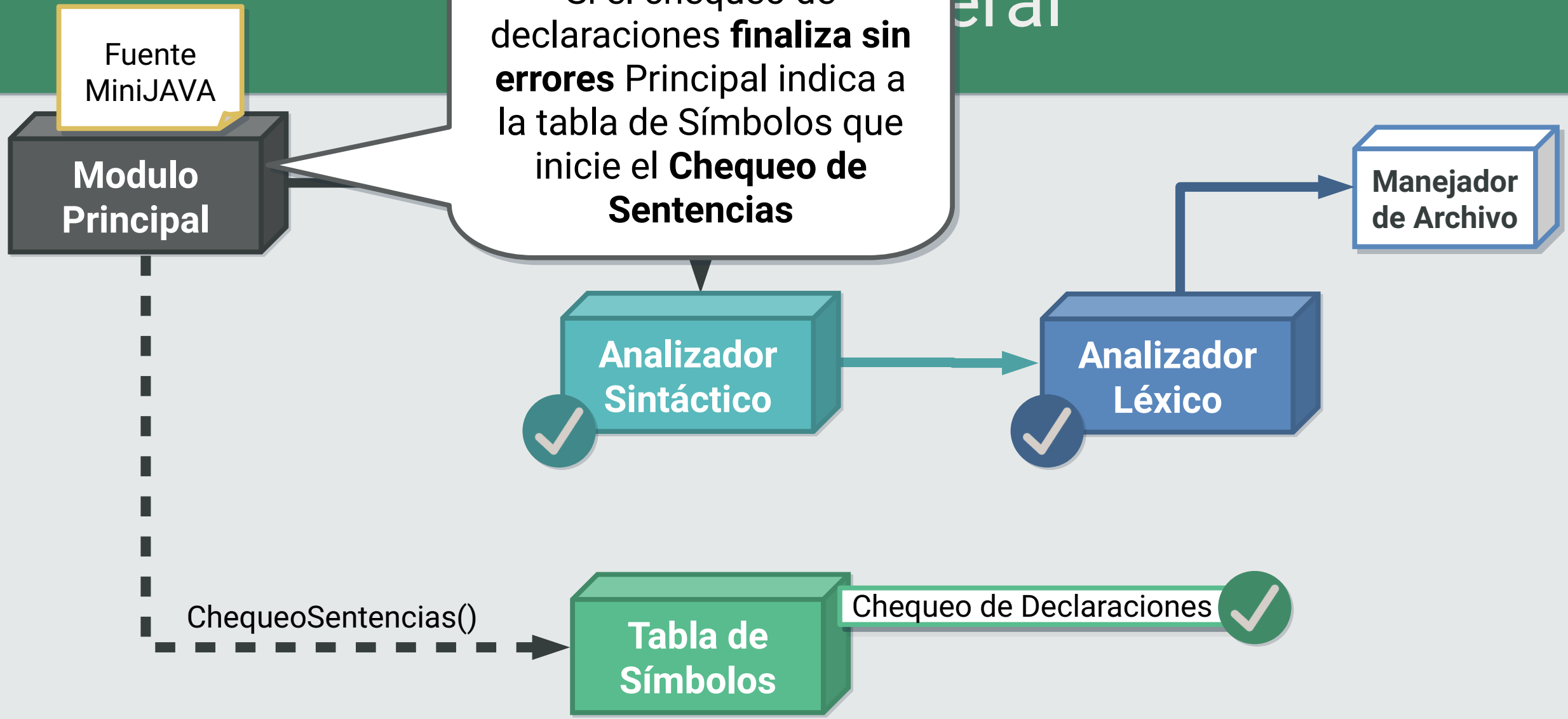
Esquema General



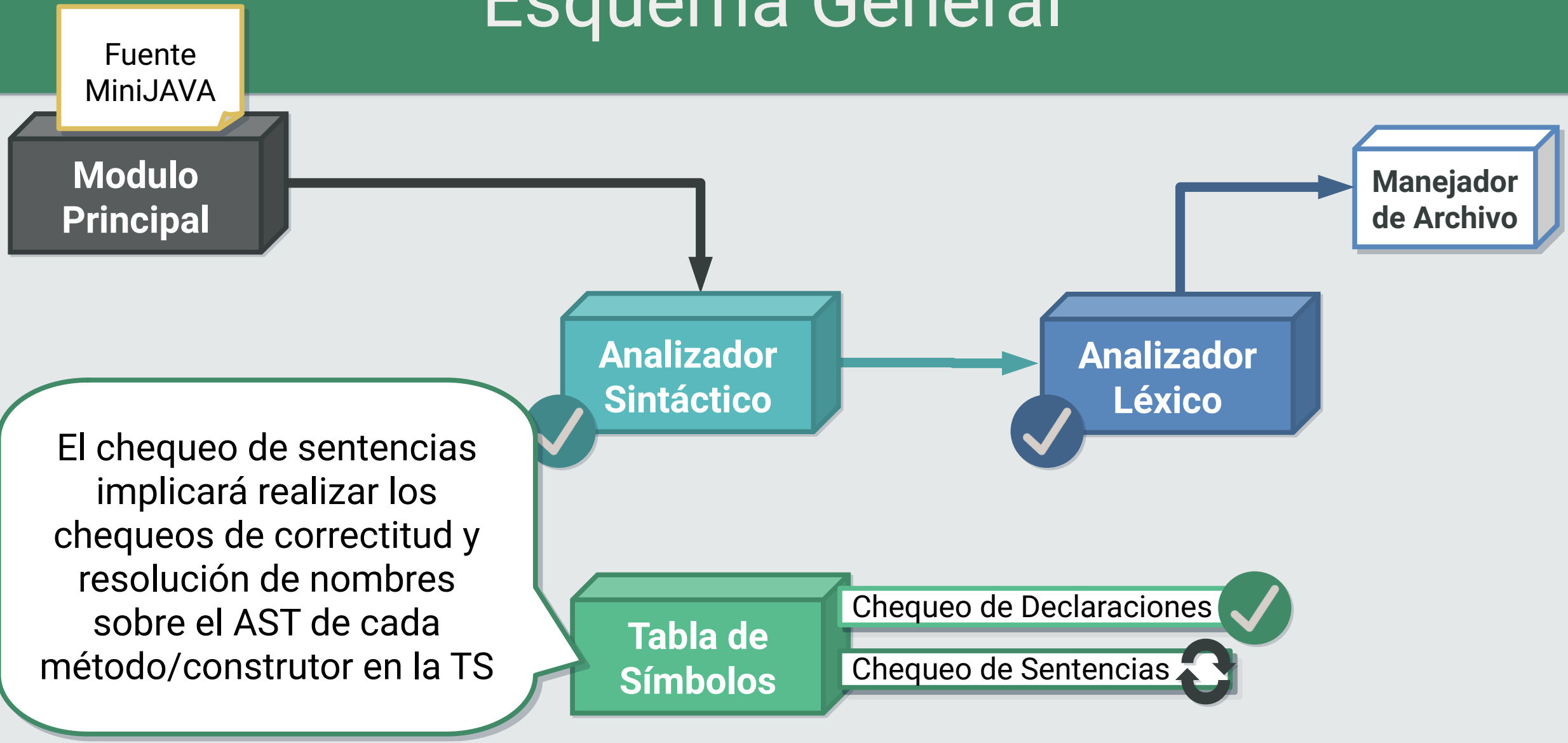
eral

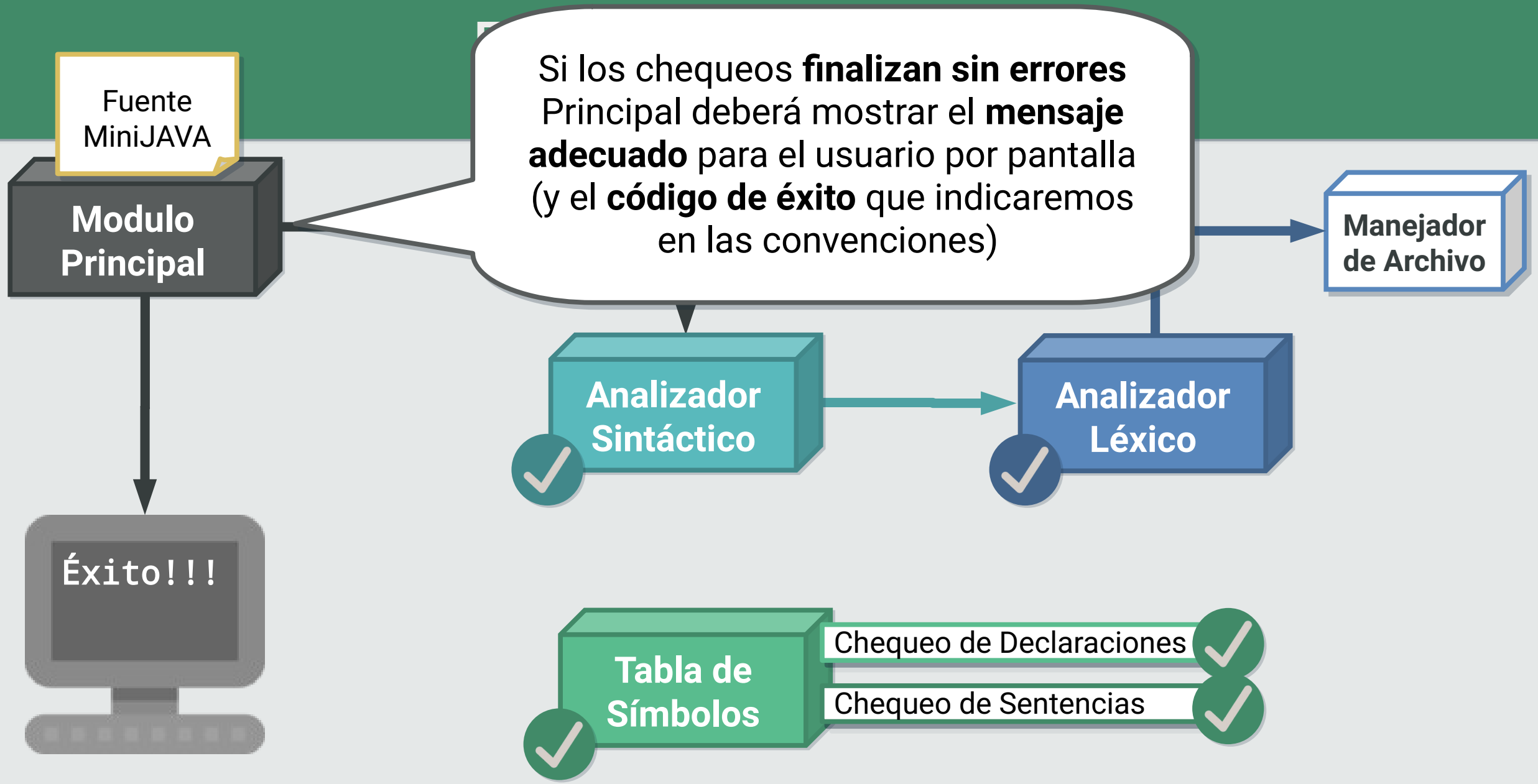

Fuente MiniJAVA
Modulo Principal
Si el **análisis sintáctico finaliza sin errores** Principal indica a la tabla de Símbolos que inicie el Chequeo de Declaraciones
Analizador Sintáctico
Analizador Léxico
Manejador de Archivo
ChequeoDeclaraciones()
Tabla de Símbolos

eral

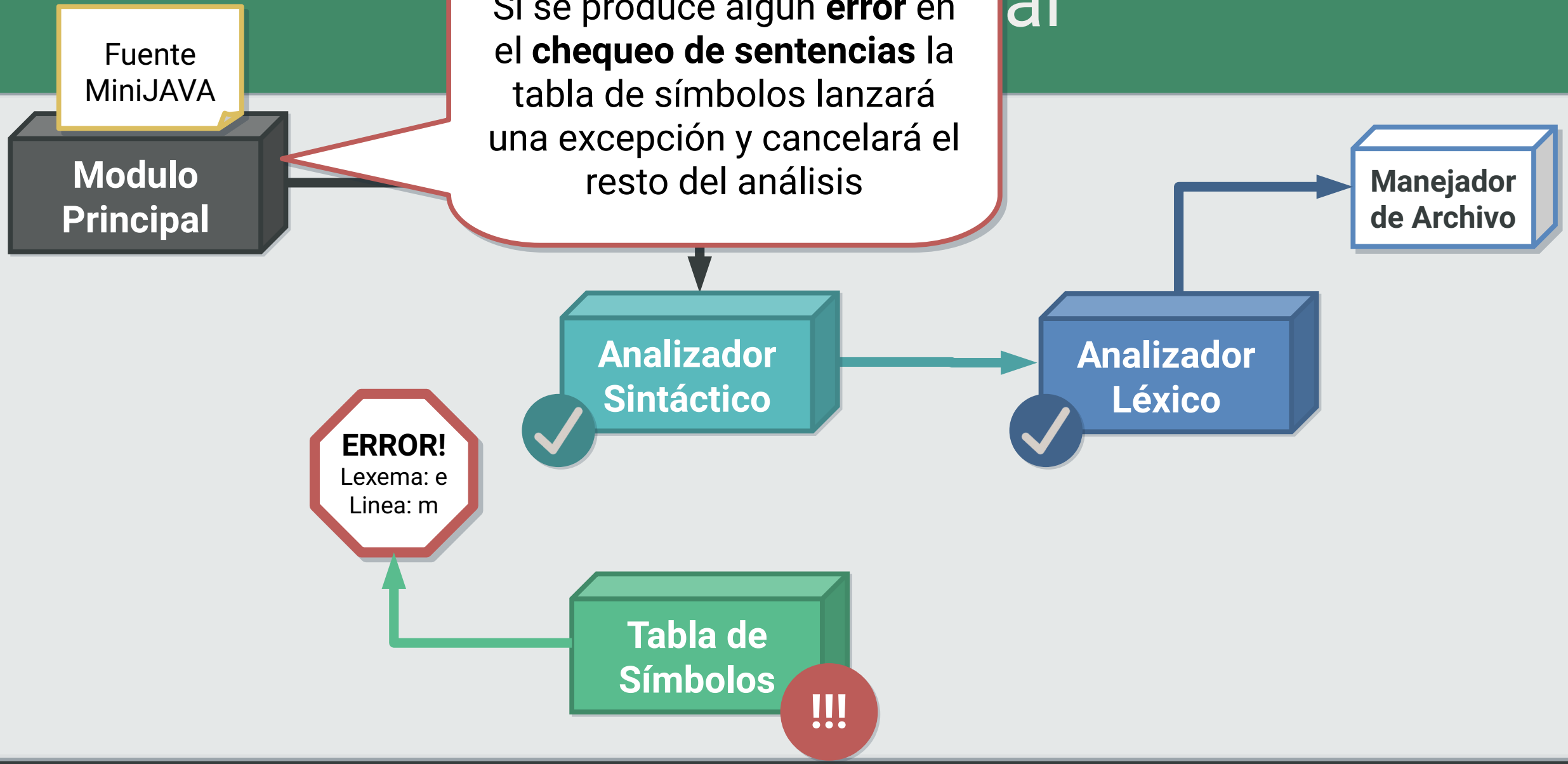

Fuente MiniJAVA
Modulo Principal
Si el chequeo de declaraciones **finaliza sin errores** Principal indica a la tabla de Símbolos que inicie el **Chequeo de Sentencias**
Analizador Sintáctico
Analizador Léxico
Manejador de Archivo
ChequeoSentencias()
Tabla de Símbolos
Chequeo de Declaraciones

Esquema General

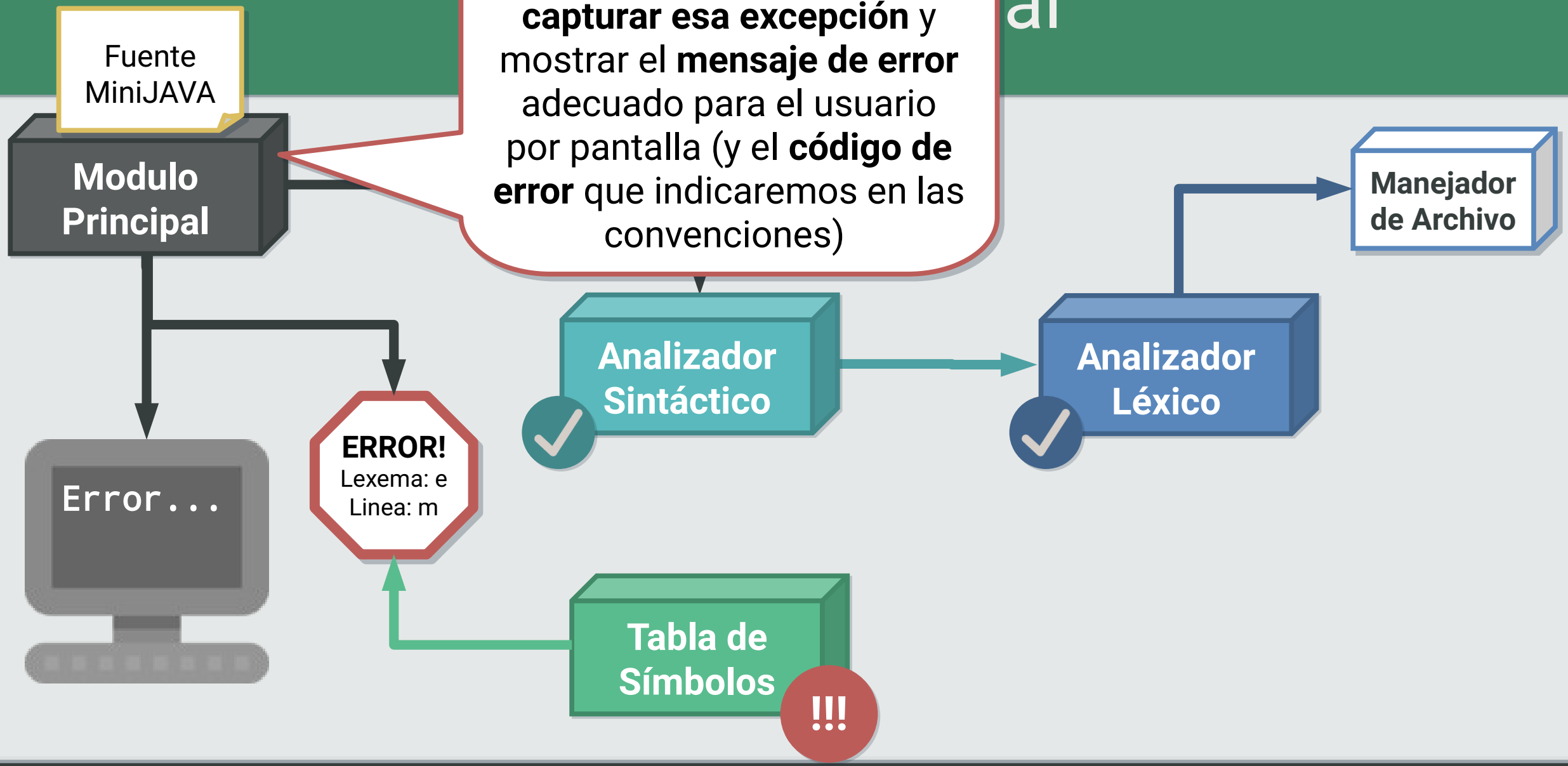



Fuente MiniJAVA
Modulo Principal
Si los chequeos **finalizan sin errores** Principal deberá mostrar el **mensaje adecuado** para el usuario por pantalla (y el **código de éxito** que indicaremos en las convenciones)
Manejador de Archivo
Analizador Sintáctico
Analizador Léxico
Éxito!!!
Tabla de Símbolos
Chequeo de Declaraciones
Chequeo de Sentencias

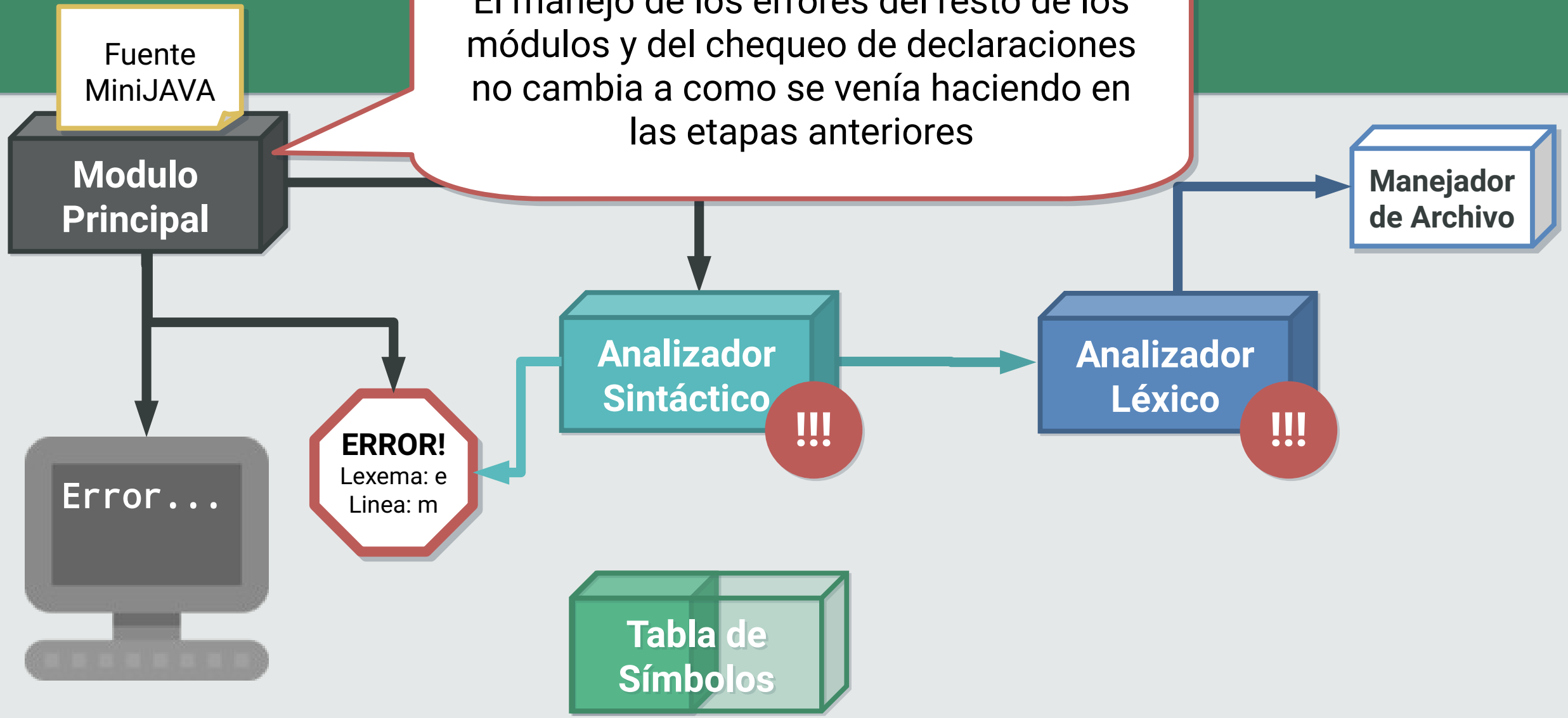
al


Fuente MiniJAVA
Modulo Principal
Si se produce algún **error** en el **chequeo de sentencias** la tabla de símbolos lanzará una excepción y cancelará el resto del análisis
Manejador de Archivo
Analizador Sintáctico
Analizador Léxico
ERROR!
Lexema: e
Linea: m
Tabla de Símbolos
!!!

...al


Fuente MiniJAVA
Modulo Principal
El modulo principal debe **capturar esa excepción** y mostrar el **mensaje de error** adecuado para el usuario por pantalla (y el **código de error** que indicaremos en las convenciones)
Analizador Sintáctico
Analizador Léxico
Manejador de Archivo
Error...
ERROR!
Lexema: e
Linea: m
Tabla de Símbolos
!!!

El manejo de los errores del resto de los módulos y del chequeo de declaraciones no cambia a como se venía haciendo en las etapas anteriores



Convenciones y Consideraciones

Convenciones y Consideraciones

Fecha Limite de Entrega: 30/10/2026 a las 16hs

Forma de Entrega: En la actividad asociada a la entrega etapa 4 en Moodle, subiendo un zip con todo lo requerido

Convenciones y Consideraciones

Reportes de Errores



- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token correspondiente a la nodo que tiene el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Ejemplo:

Fuente MiniJava

```
class A{
    void m1(){
        (4+true);
    }
}
```

El Reporte de Error debería ser

```
Error Semántico en linea 3: El tipo booleano
es incompatibles con el operador “+”

[Error:+|3]
```

Convenciones y Consideraciones

Reportes de Errores



- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token correspondiente a la nodo que tiene el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Otro Ejemplo:

Fuente MiniJava

```
class A{
    void m1(){
        a = 10;
    }
}
```

El Reporte de Error debería ser

```
Error Semántico en linea 3: La variable a no fue declarada

[Error:a|3]
```

Convenciones y Consideraciones

Mensaje de Análisis Exitoso

- Cuando no se detectan errores en el análisis el Modulo Principal debería mostrar un mensaje adecuado y luego el código de éxito: **[SinErrores]**
- Por ejemplo

Fuente MiniJava

```
class C1{

}
```

El mensaje debería ser

```
Compilacion Exitosa

[SinErrores]
```

Convenciones y Consideraciones

Invocación del Compilador



- Al ser compilado el software entregado poder invocarse de consola recibiendo como parámetro el archivo fuente MiniJava (debe aceptar cualquier tipo de extensión!)
- Por ejemplo: si al compilarlo lo llamamos Compilador y el fuente MiniJava se llama programa1.java, debería poder invocarse de la siguiente manera

```
java -jar Compilador.jar programa1.java
```

Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en **fallos** y/o a la **desaprobación** de la entrega/reentrega

Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en **fallos** y/o a la **desaprobación** de la entrega/reentrega

Las convenciones son esenciales para que **nosotros podamos corregir** bien la entrega... por eso somos tan estrictos!

Si no la podemos corregir va a estar **desaprobada!!!**

Herramientas de Test para la Evaluación

- En la tarea también les adjuntamos **herramientas** de **testing** de JUnit
- Estas herramientas las utilizamos en la cátedra como parte de la **evaluación** de los proyectos
- En el adjunto van a encontrar una **guía** de como agregarlas al proyecto (está armada respecto a **IntelliJ IDEA)**

Aclaración Importante: Las herramientas **no incluyen** los **casos de prueba** necesarios para **verificar adecuadamente** el compilador. El desarrollo de los casos es **responsabilidad del estudiante**. *Ademas, estas no son las únicas herramientas que utiliza la cátedra para la evaluación*

Informe

Informe

- El informe deberá indicar **como compilar el código fuente** entregado y como correrlo una vez compilado.
- Debe presentarse el diagrama de clases de los ASTs.
- Debe indicarse que **logros** se intentan alcanzar en la Etapa

Logros Disponibles para la Etapa 4

Logros Etapa 4



Imbatibilidad Semantica II

(no valido para la reentrega, ni etapas subsiguiente)

El software entregado pasa correctamente toda la batería de prueba utilizada por la cátedra



Minilambdas E4

Requiere: 

El compilador controla que las expresiones Lambda sean correctamente utilizadas y tipadas, como en Java. A diferencia de Java, los parámetros no pueden declarar tipos explícitamente y el cuerpo debe estar formado por una única expresión, sin llaves. No se permite la captura de variables locales o parámetros externos, ni el uso de this dentro de una Lambda.



Logros Etapa 4



Entrega Anticipada Semántica I

(no valido para la reentrega, ni etapas subsiguiente)

La entrega debe realizarse 48hs antes de la fecha limite

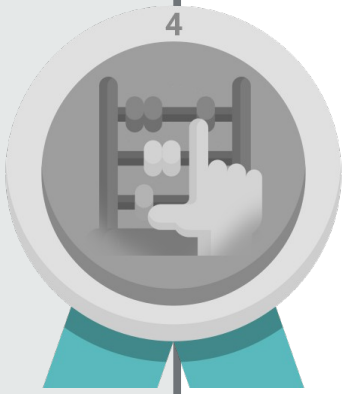


Grandioso Laburo!

(no valido para la reentrega, ni etapas subsiguiente)

Muy buen trabajo! La entrega tiene solo algunos errores menores

Logros Etapa 4

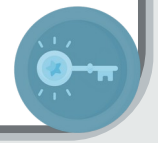


Requiere:



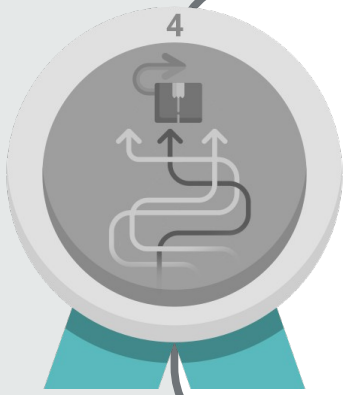
For E4

El compilador permite estructuras de repetición for y for de iteradores controlando su uso correcto. A diferencia de Java el de iteradores requiere la estructura conforme con MiniIterable<T>, una interfaz predefinida (como indica abajo). En el for standard se espera que el primer componente sea una asignacion o una declaracion var, el segundo una expresion booleana y el tercero una asignación.



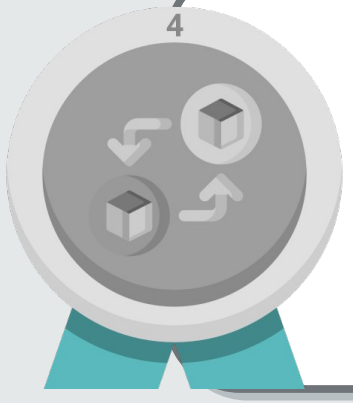
```
interface MiniIterable<T>{
    void start();
    T next();
    boolean hasNext();
}
```

Logros Etapa 4



Todos los Caminos llevan al Retorno sin codigo muerto

El compilador al igual que en Java controla que cada camino de ejecución en el cuerpo de un método llegue a una sentencia de retorno y que no haya porciones inalcanzables de código

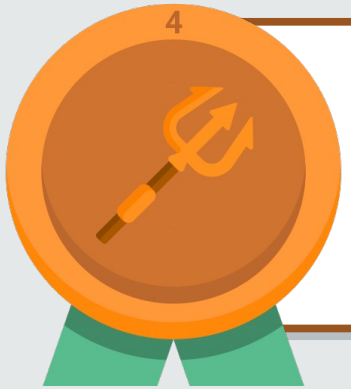


Requiere:

Floats y Coerción

El compilador controla que se usen adecuadamente los literales de tipo Float y, ademas, permite coerción. En particular, permite coerción de de ints a floats, ints a chars, chars a ints, chars a Strings y de ints a Strings. La coerción se aplica en los mismos contextos que en Java

Logros Etapa 4



Ternario E4

Requiere:

El compilador controla que las expresiones ternarias son correctamente tipadas



Genericidad Avanzada

Requiere:

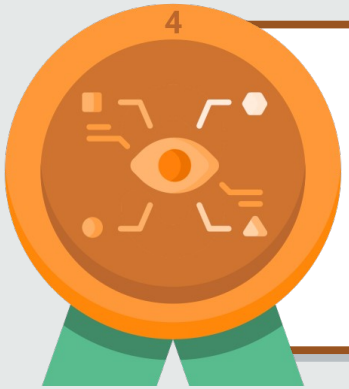
El compilador controla el uso de tipos genéricos anidados en expresiones y sentencias, y permite inferir argumentos de tipo mediante la notación diamante, como en Java. A diferencia de Java, cada clase o interfaz continúa limitada a un único parámetro de tipo. No se admiten tipos raw, comodines ni cotas. Se mantienen las reglas de conformidad de MiniJava.

Logros Etapa 4



Instanceof

El compilador permite utilizar el operador instanceof y controla su uso correcto, como en Java. A diferencia de Java, el tipo indicado a la derecha del operador debe ser una clase o interfaz no genérica. No se admiten arreglos como tipos de destino ni la declaración de una variable asociada a la prueba de tipo.



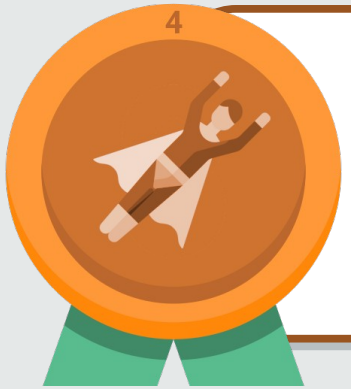
Requiere:



Visibilidad - Chequeada

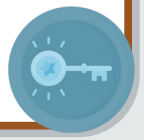
El compilador controla que los accesos sobre atributos, métodos y constructores respetan los modificadores de visibilidad como en Java

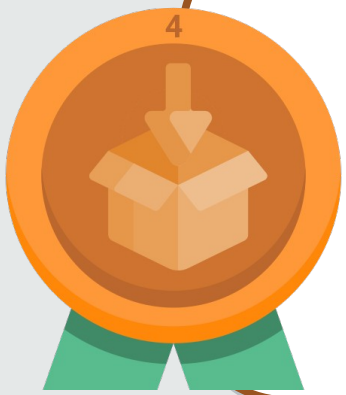
Logros Etapa 4



Super! E4

El compilador permite utilizar **super** como en Java para acceder a métodos o atributos de un ancestro, o para invocar a su constructor



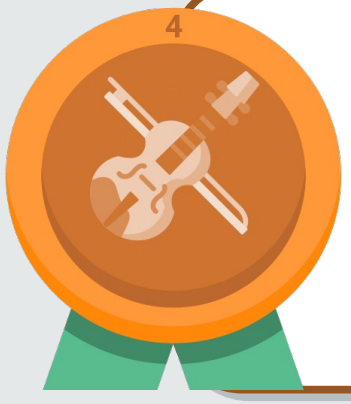


Métodos Genéricos E4

Requiere: 

El compilador controla que los métodos genéricos sean correctamente utilizados y que sus cuerpos estén correctamente tipados, como en Java. A diferencia de Java, los parámetros de tipo no tienen cotas, ni siquiera una cota implícita Object. Se mantienen las reglas de conformidad de MiniJava.

Logros Etapa 4

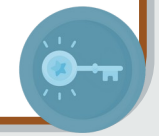


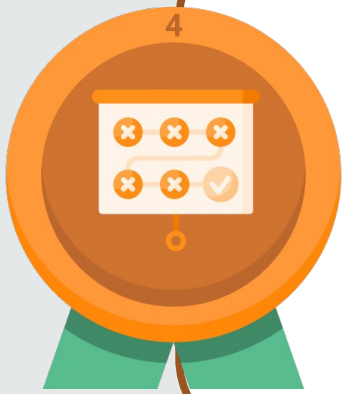
Variables Locales Clásicas Controladas

Requiere:



El compilador controla las sentencias de definición de variables clásicas (en todas sus variantes) son correctamente tipadas

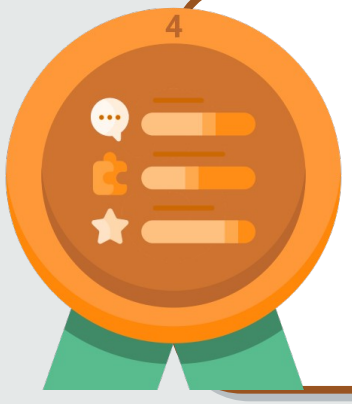




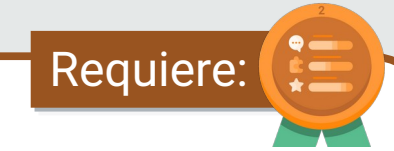
Multi-Detección Errores Semanticos en Sentencias

El compilador no finaliza la ejecución ante el primer error semántico en una sentencia o expresión (se recupera) y es capaz de reportar otros errores que tenga el programa fuente en una corrida. Como mínimo se espera que el compilador pueda continuar con el análisis de la próxima sentencia

Logros Etapa 4

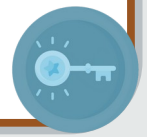


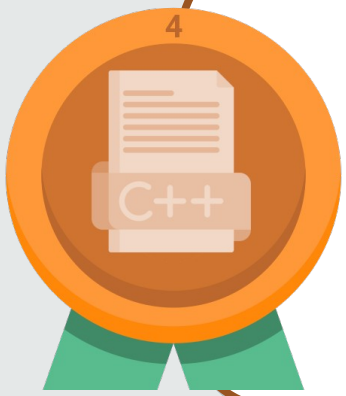
Requiere:



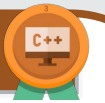
Atributos Inicializados – Chequeado!

El compilador controla que los atributos inicializados son correctamente tipados (los controles son los mismos que en Java)





Requiere:



Postfijos - Chequeado

El compilador controla que los operadores posfijos de incremento (++) y decremento (--) sean correctamente utilizados y tipados, como en Java. A diferencia de Java, solo se admiten operandos de tipo int. Se mantienen las restricciones de MiniJava sobre destinos asignables.



Logros de Etapas Anteriores

- Recordar que es posible también intentar alcanzar **logros** de la **etapa anterior**
  - **Es importante** que nos aclaren como desactivar el analizador semántico si persiguen logros del sintáctico o el léxico
- Aun así chequear que el logro de sea alcanzable en una etapa posterior ya que existen algunos que no lo permiten (como por ejemplo entrega anticipada o imbatibilidad)